Asked in 12 to 35 LPA interviews

10 SQL Patterns

SQL Server Interview Question Kit

Part 1 has one worked question for every pattern interviewers love, and Part 2 has 20 practice questions with full solutions. The patterns: running totals, top N per group, previous row comparison, gaps and islands, moving average, pivot, duplicates, window functions, self joins and recursive CTEs.

How to use this kit

1. Open SQL Server Management Studio and run the table script from the question page.
2. Read the query and predict the output on paper. Do not run it yet.
3. Pick A, B, C or D. Then run the query and compare.
4. Read the solution page. Focus on the Rule and the Pro Tip, they are what the interviewer wants to hear.

Contents

1	Running Totals	Question p.2	Solution p.12
2	Top N Per Group	Question p.3	Solution p.13
3	Previous Row Comparison	Question p.4	Solution p.14
4	Gaps and Islands	Question p.5	Solution p.15
5	Moving Average	Question p.6	Solution p.16
6	Pivot	Question p.7	Solution p.17
7	Duplications	Question p.8	Solution p.18
8	Window Functions	Question p.9	Solution p.19
9	Self Joins	Question p.10	Solution p.20
10	Recursive CTE	Question p.11	Solution p.21
+	Quick answer key	Page 22	
+	Practice set: 20 questions	Pages 23 to 32	
+	Practice set: solutions	Pages 33 to 42	

Prepared by Deepankar Sir
Instagram @datawithdeepankar | YouTube @deepankarpathak983

Can you solve this 12 to 35 LPA interview question?

Question 1 Running Totals

What does the run_total column show, from top to bottom?

Step 1: Create the table in SQL Server

```
CREATE TABLE Sales (sale_date DATE, amount INT);
INSERT INTO Sales VALUES
('2026-01-01', 100), ('2026-01-02', 200),
('2026-01-02', 300), ('2026-01-03', 50);
```

Step 2: Read this query carefully

```
SELECT sale_date, amount,
       SUM(amount) OVER (ORDER BY sale_date) AS run_total
FROM Sales;
```

Step 3: Choose your answer

A 100, 300, 600, 650

B 100, 200, 600, 650

C 100, 600, 600, 650

D 100, 600, 900, 950

Rough work

My answer: _____

Predict first. Run the query only after you have chosen. The solution is on page 12.

Can you solve this 12 to 35 LPA interview question?

Question 2 Top N Per Group

How many rows does this query return?

Step 1: Create the table in SQL Server

```
CREATE TABLE Employees (dept VARCHAR(10), emp VARCHAR(20), salary INT);
INSERT INTO Employees VALUES
('HR', 'Asha', 90), ('HR', 'Bala', 90),
('HR', 'Chirag', 80), ('HR', 'Dev', 70),
('IT', 'Esha', 95), ('IT', 'Farid', 85),
('IT', 'Gita', 85), ('IT', 'Hari', 60);
```

Step 2: Read this query carefully

```
WITH r AS (
  SELECT dept, emp, salary,
         DENSE_RANK() OVER (PARTITION BY dept ORDER BY salary DESC) AS rnk
  FROM Employees
)
SELECT dept, emp, salary FROM r WHERE rnk <= 2;
```

Step 3: Choose your answer

A 6

B 4

C 5

D 8

Rough work

My answer: _____

Predict first. Run the query only after you have chosen. The solution is on page 13.

Can you solve this 12 to 35 LPA interview question?

Question 3 Previous Row Comparison

What value does cnt hold?

Step 1: Create the table in SQL Server

```
CREATE TABLE Stock (day INT, price INT);
INSERT INTO Stock VALUES
(1, 100), (2, 110), (3, 105), (4, 105), (5, 120);
```

Step 2: Read this query carefully

```
SELECT COUNT(*) AS cnt
FROM (
  SELECT day, price,
         LAG(price) OVER (ORDER BY day) AS prev_price
  FROM Stock
) t
WHERE NOT (price > prev_price);
```

Step 3: Choose your answer

A 3

B 2

C 4

D 1

Rough work

My answer: _____

Predict first. Run the query only after you have chosen. The solution is on page 14.

Can you solve this 12 to 35 LPA interview question?

Question 4 Gaps and Islands

How many rows come back, and what is the largest value in the days column?

Step 1: Create the table in SQL Server

```
CREATE TABLE Logins (login_date DATE);
INSERT INTO Logins VALUES
('2026-03-01'), ('2026-03-02'), ('2026-03-03'),
('2026-03-05'), ('2026-03-06'), ('2026-03-08');
```

Step 2: Read this query carefully

```
SELECT MIN(login_date) AS start_dt,
       MAX(login_date) AS end_dt,
       COUNT(*)        AS days
FROM (
  SELECT login_date,
         DATEADD(DAY,
                 -ROW_NUMBER() OVER (ORDER BY login_date),
                 login_date) AS grp
  FROM Logins
) t
GROUP BY grp;
```

Step 3: Choose your answer

A 2 rows, largest days = 3

B 3 rows, largest days = 2

C 6 rows, largest days = 1

D 3 rows, largest days = 3

Rough work

My answer: _____

Predict first. Run the query only after you have chosen. The solution is on page 15.

Can you solve this 12 to 35 LPA interview question?

Question 5 Moving Average

What does avg_3m show for months 1 to 4?

Step 1: Create the table in SQL Server

```
CREATE TABLE MonthlySales (month_no INT, sales INT);
INSERT INTO MonthlySales VALUES (1, 10), (2, 25), (3, 30), (4, 45);
```

Step 2: Read this query carefully

```
SELECT month_no,
       AVG(sales) OVER (ORDER BY month_no
                        ROWS BETWEEN 2 PRECEDING AND CURRENT ROW) AS avg_3m
FROM MonthlySales;
```

Step 3: Choose your answer

A 10, 35, 65, 100

B 10, 18, 22, 33

C 10, 17, 21, 33

D 10, 17.5, 21.67, 33.33

Rough work

My answer: _____

Predict first. Run the query only after you have chosen. The solution is on page 16.

Can you solve this 12 to 35 LPA interview question?

Question 6 Pivot

What does this PIVOT query return?

Step 1: Create the table in SQL Server

```
CREATE TABLE Sales (region VARCHAR(10), sales_qtr CHAR(2), amount INT);
INSERT INTO Sales VALUES
('East', 'Q1', 100), ('East', 'Q2', 150),
('West', 'Q1', 200), ('East', 'Q1', 50);
```

Step 2: Read this query carefully

```
SELECT region, [Q1], [Q2]
FROM (SELECT region, sales_qtr, amount FROM Sales) s
PIVOT (SUM(amount) FOR sales_qtr IN ([Q1], [Q2])) p;
```

Step 3: Choose your answer

A East: 150, 150 | West: 200, NULL

B East: 150, 150 | West: 200, 0

C East: 100, 150 | West: 200, NULL

D Error: duplicate East and Q1 combination

Rough work

My answer: _____

Predict first. Run the query only after you have chosen. The solution is on page 17.

Can you solve this 12 to 35 LPA interview question?

Question 7 Duplications

Which ids remain in the Customers table after this runs?

Step 1: Create the table in SQL Server

```
CREATE TABLE Customers (id INT, email VARCHAR(50));
INSERT INTO Customers VALUES
(1, 'a@x.com'), (2, 'a@x.com'), (3, 'b@x.com'),
(4, 'a@x.com'), (5, 'b@x.com');
```

Step 2: Read this query carefully

```
WITH cte AS (
  SELECT id, email,
         ROW_NUMBER() OVER (PARTITION BY email ORDER BY id DESC) AS rn
  FROM Customers
)
DELETE FROM cte WHERE rn > 1;
```

Step 3: Choose your answer

A 1, 3

B 4, 5

C 1, 2, 3

D Error: you cannot DELETE from a CTE

Rough work

My answer: _____

Predict first. Run the query only after you have chosen. The solution is on page 18.

Can you solve this 12 to 35 LPA interview question?

Question 8 Window Functions

What does the lv column show, from top to bottom?

Step 1: Create the table in SQL Server

```
CREATE TABLE Orders (id INT, amt INT);
INSERT INTO Orders VALUES (1, 10), (2, 20), (3, 30);
```

Step 2: Read this query carefully

```
SELECT id, amt,
       LAST_VALUE(amt) OVER (ORDER BY id) AS lv
FROM Orders;
```

Step 3: Choose your answer

A 30, 30, 30

B NULL, NULL, 30

C 10, 10, 10

D 10, 20, 30

Rough work

My answer: _____

Predict first. Run the query only after you have chosen. The solution is on page 19.

Can you solve this 12 to 35 LPA interview question?

Question 9 Self Joins

How many rows does this query return?

Step 1: Create the table in SQL Server

```
CREATE TABLE Employees (emp_id INT, name VARCHAR(20), manager_id INT);
INSERT INTO Employees VALUES
(1, 'Ravi', NULL), (2, 'Sita', 1), (3, 'Amit', 1),
(4, 'Neha', 2), (5, 'Karan', 2);
```

Step 2: Read this query carefully

```
SELECT e.name AS employee, m.name AS manager
FROM Employees e
LEFT JOIN Employees m ON e.manager_id = m.emp_id
WHERE m.name <> 'Ravi';
```

Step 3: Choose your answer

A 5

B 4

C 2

D 3

Rough work

My answer: _____

Predict first. Run the query only after you have chosen. The solution is on page 20.

Can you solve this 12 to 35 LPA interview question?

Question 10 Recursive CTE

What does this query return?

Step 1: Setup

```
-- No table needed. This query builds its own rows.
```

Step 2: Read this query carefully

```
WITH cte AS (
  SELECT 1 AS n
  UNION ALL
  SELECT n * 2 FROM cte WHERE n < 20
)
SELECT SUM(n) AS total FROM cte;
```

Step 3: Choose your answer

A 63

B 31

C 127

D Error: maximum recursion level exceeded

Rough work

My answer: _____

Predict first. Run the query only after you have chosen. The solution is on page 21.

Solution 1 Running Totals

Correct answer C. 100, 600, 600, 650

What SQL Server works out

sale_date	amount	run_total
2026-01-01	100	100
2026-01-02	200	600
2026-01-02	300	600
2026-01-03	50	650

How it thinks, step by step

- You wrote ORDER BY inside OVER but no frame. SQL Server then uses the default frame: RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW.
- RANGE treats rows with the same ORDER BY value as peers. Both rows of 2026-01-02 are peers, so each one sees the other.
- For 2026-01-02 the sum is 100 + 200 + 300 = 600, and both rows show 600. For 2026-01-03 it is 600 + 50 = 650.

Why the trap fools people

Most people expect 300 and then 600 for the two tied rows (option A). That is how ROWS behaves, not RANGE. Option B just repeats the amount column, and option D adds the tied rows twice.

The Rule

ORDER BY alone means RANGE. Tied rows always get the same running total.

Pro Tip

For running totals always write ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW and add a unique tie-breaker (like an ID) to the ORDER BY. It gives the result you expect and is usually faster.

Execution order here: FROM, then SELECT (window functions are calculated here), then ORDER BY.

Solution 2 Top N Per Group

Correct answer A. 6

What SQL Server works out

dept	emp	salary
HR	Asha	90
HR	Bala	90
HR	Chirag	80
IT	Esha	95
IT	Farid	85
IT	Gita	85

How it thinks, step by step

- DENSE_RANK gives tied salaries the same rank and never skips a number. HR: 90, 90 get rank 1, 80 gets rank 2, 70 gets rank 3.
- IT: 95 gets rank 1, both 85s get rank 2, 60 gets rank 3.
- rnk <= 2 keeps the top 2 salary values, not the top 2 people. HR gives 3 rows and IT gives 3 rows, so the total is 6.

Why the trap fools people

Option B (4 rows) is what ROW_NUMBER would return. Option C (5 rows) is what RANK would return, because RANK jumps to 3 after a tie. Option D (8) is the table without any filter.

The Rule

ROW_NUMBER gives exactly N rows. RANK skips numbers after ties. DENSE_RANK never skips.

Pro Tip

Before you choose the function, ask the interviewer: top N people, or top N salary values? That one question picks ROW_NUMBER or DENSE_RANK for you.

Execution order here: WHERE runs before SELECT, so it cannot see rnk. That is why the ranking sits in a CTE and the filter sits outside.

Solution 3 Previous Row Comparison

Correct answer B. 2

What SQL Server works out

day	price	prev_price	price > prev	NOT(...)	Kept?
1	100	NULL	UNKNOWN	UNKNOWN	No
2	110	100	TRUE	FALSE	No
3	105	110	FALSE	TRUE	Yes
4	105	105	FALSE	TRUE	Yes
5	120	105	TRUE	FALSE	No

How it thinks, step by step

- LAG looks one row back. For day 1 there is no previous row, so prev_price is NULL.
- 100 > NULL is not TRUE and not FALSE. It is UNKNOWN. And NOT UNKNOWN is still UNKNOWN.
- WHERE keeps only rows that evaluate to TRUE. Day 1 is dropped. Day 3 and day 4 pass, so the count is 2.

Why the trap fools people

People read NOT (price > prev_price) as 'price is not higher', and happily count day 1 as well. That gives 3 (option A). NULL logic quietly removes that row.

The Rule

Any comparison with NULL is UNKNOWN, and NOT UNKNOWN is UNKNOWN. WHERE only keeps TRUE.

Pro Tip

Whenever you see LAG or LEAD, ask: what happens on the first row? Use LAG(price, 1, 0) for a default, or write prev_price IS NULL OR price <= prev_price to keep it on purpose.

Execution order here: FROM (inner query with LAG), then WHERE on the outer query, then SELECT COUNT.

Solution 4 Gaps and Islands

Correct answer D. 3 rows, largest days = 3

What SQL Server works out

login_date	row_number	grp (date - rn)
2026-03-01	1	2026-02-28
2026-03-02	2	2026-02-28
2026-03-03	3	2026-02-28
2026-03-05	4	2026-03-01
2026-03-06	5	2026-03-01
2026-03-08	6	2026-03-02

How it thinks, step by step

- ROW_NUMBER counts 1, 2, 3, 4, 5, 6. Subtracting it from each date gives a helper value called grp.
- While dates are consecutive, both the date and the row number go up by 1, so grp stays the same. When a date is skipped, grp changes.
- Three groups appear: Mar 1 to Mar 3 (3 days), Mar 5 to Mar 6 (2 days), and Mar 8 (1 day). So 3 rows and the largest days is 3.

Why the trap fools people

Option B is tempting if you only notice the two gaps and forget that the first island has 3 days. Always count the island sizes, not the gaps.

The Rule

Date minus ROW_NUMBER gives the island ID. Consecutive values share the same ID.

Pro Tip

This trick needs unique dates. If a date can repeat, use SELECT DISTINCT first or switch to DENSE_RANK. Add ORDER BY to the final query, because GROUP BY does not guarantee order.

Execution order here: FROM (inner query builds grp), then GROUP BY grp, then SELECT MIN, MAX, COUNT.

Solution 5 Moving Average

Correct answer C. 10, 17, 21, 33

What SQL Server works out

month_no	Rows in window	Exact average	avg_3m returned
1	10	10	10
2	10, 25	35 / 2 = 17.5	17
3	10, 25, 30	65 / 3 = 21.67	21
4	25, 30, 45	100 / 3 = 33.33	33

How it thinks, step by step

- The frame is the current row plus 2 rows before it. In months 1 and 2 there are not enough earlier rows, so the window is smaller (1 row, then 2 rows).
- The column sales is INT. In SQL Server, AVG of an INT returns an INT, so the decimal part is cut off, not rounded.
- 17.5 becomes 17, 21.67 becomes 21, and 33.33 becomes 33. Result: 10, 17, 21, 33.

Why the trap fools people

Option D is what a calculator says, but SQL Server returns the data type of the input. Option B is what you get if you round instead of truncate.

The Rule

AVG of an integer column returns an integer. Convert first if you need decimals.

Pro Tip

Write AVG(sales * 1.0) or AVG(CAST(sales AS DECIMAL(10,2))) to get decimals. To show only full 3-month windows, add COUNT(*) OVER (same frame) and keep rows where it equals 3.

Execution order here: FROM, then SELECT (window frame is calculated per row here).

Solution 6 Pivot

Correct answer A. East: 150, 150 | West: 200, NULL

What SQL Server works out

region	Q1	Q2
East	150	150
West	200	NULL

How it thinks, step by step

- PIVOT quietly does a GROUP BY on every column that is not in the PIVOT clause. Here that is only region.
- East has two Q1 rows (100 and 50), so SUM combines them into 150. East Q2 is 150.
- West has no Q2 row at all. PIVOT has nothing to sum, so the cell is NULL, not 0.

Why the trap fools people

Option D thinks duplicates cause an error. They do not, that is exactly why PIVOT needs an aggregate. Option B assumes missing values become 0, but they stay NULL.

The Rule

PIVOT = GROUP BY on the remaining columns + an aggregate, with the pivot values turned into columns.

Pro Tip

Always pivot from a small derived table with only the columns you need. An extra column (like an ID) becomes part of the grouping and breaks the result. Use ISNULL(Q2, 0) if you want zeros.

Execution order here: FROM (derived table s), then PIVOT (group and aggregate), then SELECT.

Solution 7 Duplications

Correct answer B. 4, 5

What SQL Server works out

id	email	rn	Action
4	a@x.com	1	Keep
2	a@x.com	2	Delete
1	a@x.com	3	Delete
5	b@x.com	1	Keep
3	b@x.com	2	Delete

How it thinks, step by step

- ROW_NUMBER restarts for every email (PARTITION BY) and numbers the rows from the highest id down (ORDER BY id DESC).
- So the row with the highest id in each email group gets rn = 1. Every other row gets rn of 2 or more and is deleted.
- SQL Server allows DELETE on a CTE when it reads from one table. The delete passes through to the real table. Survivors: 4 and 5.

Why the trap fools people

Option A is what you get with ORDER BY id ASC, which keeps the oldest row. Many people read the query as 'keep the first row' without checking the sort direction. Option D is wrong for this query shape.

The Rule

The ORDER BY inside ROW_NUMBER decides which duplicate survives (rn = 1).

Pro Tip

Never run the DELETE first. Run the same CTE with SELECT * WHERE rn > 1 and read the rows. Then do the delete inside BEGIN TRAN so you can ROLLBACK.

Execution order here: CTE is built first (FROM, SELECT with ROW_NUMBER), then DELETE removes the rows where rn > 1.

Solution 8 Window Functions

Correct answer D. 10, 20, 30

What SQL Server works out

id	amt	lv
1	10	10
2	20	20
3	30	30

How it thinks, step by step

- With ORDER BY and no frame, the default frame is RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW.
- So for each row, the window ends at the current row. Row 1 sees only row 1, row 2 sees rows 1 and 2, row 3 sees all three.
- The last value inside each of those windows is the current row itself. So lv equals amt: 10, 20, 30.

Why the trap fools people

The name LAST_VALUE tricks you into reading it as 'the last value of the whole table' (option A). It is only the last value of the frame, and the default frame stops at the current row.

The Rule

LAST_VALUE needs an explicit frame to reach the end: ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING.

Pro Tip

Whenever you use LAST_VALUE, write the frame out every time. Or flip the order and use FIRST_VALUE with ORDER BY id DESC, which works with the default frame.

Execution order here: FROM, then SELECT (the frame is applied per row), then output.

Solution 9 Self Joins

Correct answer C. 2

What SQL Server works out

employee	manager	m.name <> 'Ravi'	Kept?
Ravi	NULL	UNKNOWN	No
Sita	Ravi	FALSE	No
Amit	Ravi	FALSE	No
Neha	Sita	TRUE	Yes
Karan	Sita	TRUE	Yes

How it thinks, step by step

- The LEFT JOIN runs first and returns all 5 employees. Ravi has no manager, so his manager column is NULL.
- Then WHERE runs on the joined result. For Ravi, NULL <> 'Ravi' is UNKNOWN, so his row is removed.
- Sita and Amit report to Ravi, so they are removed too. Only Neha and Karan are left: 2 rows.

Why the trap fools people

The LEFT JOIN was written to keep rows without a match, but the WHERE filter on the right table removes exactly those rows. Option D (3 rows) assumes Ravi survives.

The Rule

A WHERE filter on the right side of a LEFT JOIN throws away the NULL rows and turns it into an INNER JOIN.

Pro Tip

If you want Ravi to stay, write WHERE m.name IS NULL OR m.name <> 'Ravi', which returns 3 rows. Put a condition in ON only when it should control the matching, not the final filtering.

Execution order here: FROM and JOIN first, then WHERE, then SELECT.

Solution 10 Recursive CTE

Correct answer A. 63

What SQL Server works out

Step	n going in	n < 20 ?	New row
Anchor	-	-	1
1	1	Yes	2
2	2	Yes	4
3	4	Yes	8
4	8	Yes	16
5	16	Yes	32
6	32	No	stop

How it thinks, step by step

- The anchor gives 1. Each round takes the rows from the previous round, checks n < 20, and doubles them.
- The check looks at the row going in, not at the row coming out. 16 < 20 is true, so 16 still produces 32. Then 32 < 20 is false, so the recursion stops.
- Rows: 1, 2, 4, 8, 16, 32. The sum is 63. Only 5 recursion rounds ran, well under the default limit of 100.

Why the trap fools people

Option B (31) assumes it stops at 16. Option C (127) assumes it goes on to 64. Option D is wrong because the limit of 100 levels is never reached here.

The Rule

In a recursive CTE the WHERE condition tests the row going in, so the last row produced can already cross the limit.

Pro Tip

Trace the anchor, the first two rounds and the last two rounds on paper. For deep recursion use OPTION (MAXRECURSION 500), and 0 for no limit, but be careful about infinite loops.

Execution order here: Anchor runs once, the recursive part repeats until it returns no rows, then the outer SELECT runs.

Quick Answer Key *Revise in 2 minutes*

#	Pattern	Answer	One-line rule
1	Running Totals	**C**. 100, 600, 600, 650	ORDER BY alone means RANGE. Tied rows always get the same running total.
2	Top N Per Group	**A**. 6	ROW_NUMBER gives exactly N rows. RANK skips numbers after ties. DENSE_RANK never skips.
3	Previous Row Comparison	**B**. 2	Any comparison with NULL is UNKNOWN, and NOT UNKNOWN is UNKNOWN. WHERE only keeps TRUE.
4	Gaps and Islands	**D**. 3 rows, largest days = 3	Date minus ROW_NUMBER gives the island ID. Consecutive values share the same ID.
5	Moving Average	**C**. 10, 17, 21, 33	AVG of an integer column returns an integer. Convert first if you need decimals.
6	Pivot	**A**. East: 150, 150 | West: 200, NULL	PIVOT = GROUP BY on the remaining columns + an aggregate, with the pivot values turned into columns.
7	Duplications	**B**. 4, 5	The ORDER BY inside ROW_NUMBER decides which duplicate survives (rn = 1).
8	Window Functions	**D**. 10, 20, 30	LAST_VALUE needs an explicit frame to reach the end: ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING.
9	Self Joins	**C**. 2	A WHERE filter on the right side of a LEFT JOIN throws away the NULL rows and turns it into an INNER JOIN.
10	Recursive CTE	**A**. 63	In a recursive CTE the WHERE condition tests the row going in, so the last row produced can already cross the limit.

Before your next interview

Most of these questions are not hard SQL. They are small traps: NULL logic, default window frames, data types and the order in which SQL Server runs each clause. Practise explaining the Rule in your own words and you will stand out from candidates who only memorise syntax.

Follow @datawithdeepankar on Instagram for a new SQL Server interview question every day. Questions about this kit? Send a DM.

Practice 1 Running Totals

A wallet app stores every transaction. Show each transaction with the running balance after it. Sort by txn_date, then txn_id.

Table script

```
CREATE TABLE WalletTxn (txn_id INT, txn_date DATE, amount INT);
INSERT INTO WalletTxn VALUES
(1, '2026-04-01', 500), (2, '2026-04-02', -200),
(3, '2026-04-03', 300), (4, '2026-04-03', -100),
(5, '2026-04-05', 400);
```

Hint: Write the ROWS frame yourself and add a tie-breaker.

Write your query here

Practice 2 Running Totals

Show each order with the running total of that customer's orders. Sort by customer, then order_date.

Table script

```
CREATE TABLE CustOrders (customer VARCHAR(10), order_date DATE, amt INT);
INSERT INTO CustOrders VALUES
('Asha', '2026-01-05', 200), ('Asha', '2026-01-09', 300),
('Ravi', '2026-01-06', 150), ('Asha', '2026-01-15', 100),
('Ravi', '2026-01-20', 250);
```

Hint: The total must restart for each customer.

Write your query here

Practice 3 Top N Per Group

Find the top 2 selling products in each category by units_sold. If two products tie for 2nd place, show both. Sort by category, units_sold DESC, product.

Table script

```
CREATE TABLE ProductSales (category VARCHAR(15), product VARCHAR(15), units_sold INT);
INSERT INTO ProductSales VALUES
('Electronics', 'Phone', 120), ('Electronics', 'Laptop', 90),
('Electronics', 'Earbuds', 150), ('Electronics', 'Tablet', 90),
('Grocery', 'Rice', 300), ('Grocery', 'Oil', 250),
('Grocery', 'Tea', 250), ('Grocery', 'Salt', 100);
```

Hint: Which ranking function gives tied rows the same rank?

Write your query here

Practice 4 Top N Per Group

Return exactly one row per department: the highest paid employee. If salaries tie, pick the lower emp_id. Sort by dept.

Table script

```
CREATE TABLE StaffPay (emp_id INT, dept VARCHAR(10), salary INT);
INSERT INTO StaffPay VALUES
(1, 'Sales', 50), (2, 'Sales', 70), (3, 'Sales', 70),
(4, 'Tech', 90), (5, 'Tech', 80), (6, 'Tech', 90);
```

Hint: You need exactly one row per group, even with ties.

Write your query here

Practice 5 Previous Row Comparison

Show every day whose revenue is higher than the previous day, along with the previous day's revenue. Sort by sale_date.

Table script

```
CREATE TABLE DailySales (sale_date DATE, revenue INT);
INSERT INTO DailySales VALUES
('2026-04-01', 1000), ('2026-04-02', 1200), ('2026-04-03', 900),
('2026-04-04', 900), ('2026-04-05', 1500);
```

Hint: LAG cannot be used directly in WHERE.

Write your query here

Practice 6 Previous Row Comparison

For every visit, show the number of days until the same user's next visit. A user's last visit has no next visit, so show NULL. Sort by user_id, visit_date.

Table script

```
CREATE TABLE SiteVisits (user_id INT, visit_date DATE);
INSERT INTO SiteVisits VALUES
(1, '2026-03-01'), (1, '2026-03-04'), (1, '2026-03-10'),
(2, '2026-03-02'), (2, '2026-03-03');
```

Hint: Use the sibling of LAG that looks forward.

Write your query here

Practice 7 Gaps and Islands

Some invoice numbers are missing. List every run of consecutive invoice numbers with its start, its end and how many invoices it has. Sort by start.

Table script

```
CREATE TABLE Invoices (invoice_no INT);
INSERT INTO Invoices VALUES
(1), (2), (3), (7), (8), (10), (11), (12);
```

Hint: Number minus ROW_NUMBER stays the same inside a run.

Write your query here

Practice 8 Gaps and Islands

Some users have more than one activity on the same day. For each user find the longest streak of consecutive active days. Count each day only once. Sort by user_id.

Table script

```
CREATE TABLE UserActivity (user_id INT, activity_date DATE);
INSERT INTO UserActivity VALUES
(1, '2026-06-01'), (1, '2026-06-02'), (1, '2026-06-02'),
(1, '2026-06-03'), (1, '2026-06-07'), (1, '2026-06-08'),
(2, '2026-06-05'), (2, '2026-06-06');
```

Hint: Remove the repeated days before you build the groups.

Write your query here

Practice 9 Moving Average

Show the 3-day moving average of temp (the day and the 2 days before it) only for days that have a full 3-day window. Round to 2 decimals. Sort by day_no.

Table script

```
CREATE TABLE CityTemps (day_no INT, temp DECIMAL(5,1));
INSERT INTO CityTemps VALUES
(1, 30.0), (2, 32.0), (3, 31.0), (4, 35.0), (5, 34.0);
```

Hint: COUNT with the same frame tells you how many rows are in the window.

Write your query here

Practice 10 Moving Average

Show the 3-month moving average of cost (current month plus the 2 months before it), rounded to 2 decimals. Include months 1 and 2, which have smaller windows. Sort by month_no.

Table script

```
CREATE TABLE MonthlyCost (month_no INT, cost INT);
INSERT INTO MonthlyCost VALUES (1, 10), (2, 15), (3, 20), (4, 40);
```

Hint: cost is an INT column. Remember what AVG does with integers.

Write your query here

Practice 11 Pivot

Show one row per product with revenue for Q1, Q2 and Q3 as separate columns. Use PIVOT. Show 0 where there is no revenue. Sort by product.

Table script

```
CREATE TABLE QuarterSales (product VARCHAR(10), qtr CHAR(2), revenue INT);
INSERT INTO QuarterSales VALUES
('Pen', 'Q1', 100), ('Pen', 'Q2', 120), ('Pen', 'Q1', 50),
('Book', 'Q2', 300), ('Book', 'Q3', 200);
```

Hint: Missing combinations come out as NULL. Fix them in the outer SELECT.

Write your query here

Practice 12 Pivot

Show one row per city with the number of Delivered, Cancelled and Pending orders as separate columns. Do not use PIVOT, use CASE. Sort by city.

Table script

```
CREATE TABLE CityOrders (order_id INT, city VARCHAR(10), status VARCHAR(10));
INSERT INTO CityOrders VALUES
(1, 'Jaipur', 'Delivered'), (2, 'Jaipur', 'Delivered'),
(3, 'Jaipur', 'Cancelled'), (4, 'Delhi', 'Delivered'),
(5, 'Delhi', 'Cancelled'), (6, 'Delhi', 'Cancelled'),
(7, 'Delhi', 'Pending');
```

Hint: Add 1 when the status matches, otherwise add 0.

Write your query here

Practice 13 Duplications

Find every email that appears more than once, with how many times it appears. Sort by email.

Table script

```
CREATE TABLE AppUsers (user_id INT, email VARCHAR(30));
INSERT INTO AppUsers VALUES
(1, 'a@x.com'), (2, 'b@x.com'), (3, 'a@x.com'),
(4, 'c@x.com'), (5, 'b@x.com'), (6, 'a@x.com');
```

Hint: Which clause filters on a group count?

Write your query here

Practice 14 Duplications

Several contacts share the same phone. Delete the duplicates but keep the oldest row (earliest created_on) for each phone. Then show the rows that remain, sorted by contact_id.

Table script

```
CREATE TABLE Contacts (contact_id INT, phone VARCHAR(5), created_on DATE);
INSERT INTO Contacts VALUES
(1, '900', '2026-01-01'), (2, '900', '2026-02-01'),
(3, '800', '2026-01-15'), (4, '900', '2026-01-10'),
(5, '800', '2026-03-01');
```

Hint: Number the rows per phone, then delete the ones numbered above 1.

Write your query here

Practice 15 Window Functions

Show each employee's salary, the average salary of their department and the difference between the two. Sort by emp.

Table script

```
CREATE TABLE TeamSalary (emp VARCHAR(5), dept VARCHAR(5), salary INT);
INSERT INTO TeamSalary VALUES
('A', 'IT', 60), ('B', 'IT', 80), ('C', 'HR', 50),
('D', 'HR', 70), ('E', 'HR', 90);
```

Hint: You need every row, so GROUP BY will not work here.

Write your query here

Practice 16 Window Functions

Show each student with the topper's marks of their class (the highest marks in that class). Do not use MAX or GROUP BY, use LAST_VALUE. Sort by class, marks.

Table script

```
CREATE TABLE ClassMarks (student VARCHAR(5), class CHAR(1), marks INT);
INSERT INTO ClassMarks VALUES
('Ana', 'X', 70), ('Ben', 'X', 90), ('Cia', 'X', 80),
('Dev', 'Y', 60), ('Eli', 'Y', 75);
```

Hint: The default frame stops at the current row. Change it.

Write your query here

Practice 17 Self Joins

List every employee with the name of their manager. Show 'No Manager' for the top boss. Sort by emp_id.

Table script

```
CREATE TABLE OrgChart (emp_id INT, emp_name VARCHAR(10), manager_id INT);
INSERT INTO OrgChart VALUES
(1, 'Meera', NULL), (2, 'Karan', 1), (3, 'Zoya', 1),
(4, 'Ishaan', 2), (5, 'Pooja', 3);
```

Hint: The boss has no match, so an INNER JOIN would lose that row.

Write your query here

Practice 18 Self Joins

Find employees who earn more than their own manager. Show the employee, their salary, the manager and the manager's salary. Sort by emp_id.

Table script

```
CREATE TABLE StaffBoss (emp_id INT, name VARCHAR(10), salary INT, manager_id INT);
INSERT INTO StaffBoss VALUES
(1, 'Asha', 100, NULL), (2, 'Bala', 120, 1), (3, 'Chirag', 80, 1),
(4, 'Dev', 90, 2), (5, 'Esha', 130, 2);
```

Hint: Compare e.salary with m.salary inside the join condition.

Write your query here

Practice 19 Recursive CTE

Without using a calendar table, generate every date from 2026-05-01 to 2026-05-05, one row per date.

Table script

```
-- No table needed. The query builds its own rows.
```

Hint: Start with one date and keep adding a day until you reach the end date.

Write your query here

Practice 20 Recursive CTE

Show each person's level in the company (the top boss is level 1) and the full reporting path from the top boss. Sort by level, then emp_id.

Table script

```
CREATE TABLE OrgTree (emp_id INT, name VARCHAR(10), manager_id INT);
INSERT INTO OrgTree VALUES
(1, 'Meera', NULL), (2, 'Karan', 1), (3, 'Zoya', 1),
(4, 'Ishaan', 2), (5, 'Pooja', 4);
```

Hint: Start from the row where manager_id IS NULL, then join each level to the one above.

Write your query here

Solution 1 Running Totals

```sql
SELECT txn_id, txn_date, amount,
       SUM(amount) OVER (ORDER BY txn_date, txn_id
                         ROWS BETWEEN UNBOUNDED PRECEDING
                              AND CURRENT ROW) AS running_balance
FROM WalletTxn
ORDER BY txn_date, txn_id;
```

Expected output

txn_id	txn_date	amount	running_balance
1	2026-04-01	500	500
2	2026-04-02	-200	300
3	2026-04-03	300	600
4	2026-04-03	-100	500
5	2026-04-05	400	900

Key idea: Ordering by txn_date and txn_id removes the tie, and ROWS makes each row add only itself.

Solution 2 Running Totals

```sql
SELECT customer, order_date, amt,
       SUM(amt) OVER (PARTITION BY customer
                      ORDER BY order_date
                      ROWS BETWEEN UNBOUNDED PRECEDING
                           AND CURRENT ROW) AS running_total
FROM CustOrders
ORDER BY customer, order_date;
```

Expected output

customer	order_date	amt	running_total
Asha	2026-01-05	200	200
Asha	2026-01-09	300	500
Asha	2026-01-15	100	600
Ravi	2026-01-06	150	150
Ravi	2026-01-20	250	400

Key idea: PARTITION BY restarts the running total for every customer.

Solution 3 Top N Per Group

```sql
WITH r AS (
  SELECT category, product, units_sold,
         RANK() OVER (PARTITION BY category
                      ORDER BY units_sold DESC) AS rnk
  FROM ProductSales
)
SELECT category, product, units_sold
FROM r
WHERE rnk <= 2
ORDER BY category, units_sold DESC, product;
```

Expected output

category	product	units_sold
Electronics	Earbuds	150
Electronics	Phone	120
Grocery	Rice	300
Grocery	Oil	250
Grocery	Tea	250

Key idea: RANK keeps both tied products at rank 2. Filter the rank in an outer query, because WHERE cannot see it.

Solution 4 Top N Per Group

```sql
WITH r AS (
  SELECT emp_id, dept, salary,
         ROW_NUMBER() OVER (PARTITION BY dept
                            ORDER BY salary DESC, emp_id) AS rn
  FROM StaffPay
)
SELECT emp_id, dept, salary
FROM r
WHERE rn = 1
ORDER BY dept;
```

Expected output

emp_id	dept	salary
2	Sales	70
4	Tech	90

Key idea: ROW_NUMBER never gives two rows the same number, and emp_id in the ORDER BY settles the tie.

Solution 5 Previous Row Comparison

```sql
WITH d AS (
  SELECT sale_date, revenue,
         LAG(revenue) OVER (ORDER BY sale_date) AS prev_revenue
  FROM DailySales
)
SELECT sale_date, revenue, prev_revenue
FROM d
WHERE revenue > prev_revenue
ORDER BY sale_date;
```

Expected output

sale_date	revenue	prev_revenue
2026-04-02	1200	1000
2026-04-05	1500	900

Key idea: The first day has a NULL previous value, so the comparison is UNKNOWN and that row drops out.

Solution 6 Previous Row Comparison

```sql
SELECT user_id, visit_date,
       DATEDIFF(DAY, visit_date,
                LEAD(visit_date) OVER (PARTITION BY user_id
                                       ORDER BY visit_date)) AS days_to_next
FROM SiteVisits
ORDER BY user_id, visit_date;
```

Expected output

user_id	visit_date	days_to_next
1	2026-03-01	3
1	2026-03-04	6
1	2026-03-10	NULL
2	2026-03-02	1
2	2026-03-03	NULL

Key idea: LEAD looks at the next row inside each user. DATEDIFF turns the two dates into a day count.

Solution 7 Gaps and Islands

```sql
SELECT MIN(invoice_no) AS start_no,
       MAX(invoice_no) AS end_no,
       COUNT(*)        AS total
FROM (
  SELECT invoice_no,
         invoice_no - ROW_NUMBER() OVER (ORDER BY invoice_no) AS grp
  FROM Invoices
) t
GROUP BY grp
ORDER BY start_no;
```

Expected output

start_no	end_no	total
1	3	3
7	8	2
10	12	3

Key idea: Inside a run, the number and the row number both rise by 1, so their difference is constant.

Solution 8 Gaps and Islands

```sql
WITH d AS (
  SELECT DISTINCT user_id, activity_date
  FROM UserActivity
),
g AS (
  SELECT user_id, activity_date,
         DATEADD(DAY,
                 -ROW_NUMBER() OVER (PARTITION BY user_id
                                     ORDER BY activity_date),
                 activity_date) AS grp
  FROM d
),
s AS (
  SELECT user_id, grp, COUNT(*) AS streak
  FROM g
  GROUP BY user_id, grp
)
SELECT user_id, MAX(streak) AS longest_streak
FROM s
GROUP BY user_id
ORDER BY user_id;
```

Expected output

user_id	longest_streak
1	3
2	2

Key idea: Duplicate dates break the date minus ROW_NUMBER trick, so DISTINCT comes first. Then group twice: islands, then the longest per user.

Solution 9 Moving Average

```sql
WITH m AS (
  SELECT day_no, temp,
         AVG(temp) OVER (ORDER BY day_no
                         ROWS BETWEEN 2 PRECEDING AND CURRENT ROW) AS avg_3d,
         COUNT(*)  OVER (ORDER BY day_no
                         ROWS BETWEEN 2 PRECEDING AND CURRENT ROW) AS cnt
  FROM CityTemps
)
SELECT day_no, temp, CAST(avg_3d AS DECIMAL(5,2)) AS avg_3d
FROM m
WHERE cnt = 3
ORDER BY day_no;
```

Expected output

day_no	temp	avg_3d
3	31.0	31.00
4	35.0	32.67
5	34.0	33.33

Key idea: A window with fewer than 3 rows has cnt below 3, so the filter keeps only complete windows.

Solution 10 Moving Average

```sql
SELECT month_no, cost,
       CAST(AVG(cost * 1.0) OVER (ORDER BY month_no
                                  ROWS BETWEEN 2 PRECEDING
                                       AND CURRENT ROW)
            AS DECIMAL(10,2)) AS avg_3m
FROM MonthlyCost
ORDER BY month_no;
```

Expected output

month_no	cost	avg_3m
1	10	10.00
2	15	12.50
3	20	15.00
4	40	25.00

Key idea: Multiplying by 1.0 turns the integers into decimals, so AVG keeps the fraction (12.50 instead of 12).

Solution 11 Pivot

```
SELECT product,
       ISNULL([Q1], 0) AS Q1,
       ISNULL([Q2], 0) AS Q2,
       ISNULL([Q3], 0) AS Q3
FROM (SELECT product, qtr, revenue FROM QuarterSales) s
PIVOT (SUM(revenue) FOR qtr IN ([Q1], [Q2], [Q3])) p
ORDER BY product;
```

Expected output

product	Q1	Q2	Q3
Book	0	300	200
Pen	150	120	0

Key idea: PIVOT sums the two Pen Q1 rows into 150. ISNULL replaces the missing cells with 0.

Solution 12 Pivot

```
SELECT city,
       SUM(CASE WHEN status = 'Delivered' THEN 1 ELSE 0 END) AS delivered,
       SUM(CASE WHEN status = 'Cancelled' THEN 1 ELSE 0 END) AS cancelled,
       SUM(CASE WHEN status = 'Pending'   THEN 1 ELSE 0 END) AS pending
FROM CityOrders
GROUP BY city
ORDER BY city;
```

Expected output

city	delivered	cancelled	pending
Delhi	1	2	1
Jaipur	2	1	0

Key idea: Conditional aggregation does the same job as PIVOT, works on every database and gives 0 instead of NULL.

Solution 13 Duplications

```sql
SELECT email, COUNT(*) AS times_used
FROM AppUsers
GROUP BY email
HAVING COUNT(*) > 1
ORDER BY email;
```

Expected output

email	times_used
a@x.com	3
b@x.com	2

Key idea: WHERE filters rows before grouping. HAVING filters the groups after COUNT is known.

Solution 14 Duplications

```sql
WITH c AS (
  SELECT contact_id,
         ROW_NUMBER() OVER (PARTITION BY phone
                            ORDER BY created_on, contact_id) AS rn
  FROM Contacts
)
DELETE FROM c WHERE rn > 1;

SELECT contact_id, phone, created_on
FROM Contacts
ORDER BY contact_id;
```

Expected output

contact_id	phone	created_on
1	900	2026-01-01
3	800	2026-01-15

Key idea: Sorting by created_on puts the oldest row at rn = 1. Everything else is deleted through the CTE.

Solution 15 Window Functions

```
SELECT emp, dept, salary,
       AVG(salary) OVER (PARTITION BY dept) AS dept_avg,
       salary - AVG(salary) OVER (PARTITION BY dept) AS diff
FROM TeamSalary
ORDER BY emp;
```

Expected output

emp	dept	salary	dept_avg	diff
A	IT	60	70	-10
B	IT	80	70	10
C	HR	50	70	-20
D	HR	70	70	0
E	HR	90	70	20

Key idea: A window aggregate adds the group result to every row without collapsing the rows.

Solution 16 Window Functions

```
SELECT student, class, marks,
       LAST_VALUE(marks) OVER (PARTITION BY class
                               ORDER BY marks
                               ROWS BETWEEN UNBOUNDED PRECEDING
                                    AND UNBOUNDED FOLLOWING) AS topper_marks
FROM ClassMarks
ORDER BY class, marks;
```

Expected output

student	class	marks	topper_marks
Ana	X	70	90
Cia	X	80	90
Ben	X	90	90
Dev	Y	60	75
Eli	Y	75	75

Key idea: With the full frame, the last row of every class (highest marks) is visible to every row of that class.

Solution 17 Self Joins

```sql
SELECT e.emp_id,
       e.emp_name AS employee,
       ISNULL(m.emp_name, 'No Manager') AS manager
FROM OrgChart e
LEFT JOIN OrgChart m ON e.manager_id = m.emp_id
ORDER BY e.emp_id;
```

Expected output

emp_id	employee	manager
1	Meera	No Manager
2	Karan	Meera
3	Zoya	Meera
4	Ishaan	Karan
5	Pooja	Zoya

Key idea: The same table is used twice with two aliases. LEFT JOIN keeps the boss, ISNULL fills in the label.

Solution 18 Self Joins

```sql
SELECT e.emp_id,
       e.name   AS employee, e.salary AS emp_salary,
       m.name   AS manager,  m.salary AS mgr_salary
FROM StaffBoss e
JOIN StaffBoss m ON e.manager_id = m.emp_id
WHERE e.salary > m.salary
ORDER BY e.emp_id;
```

Expected output

emp_id	employee	emp_salary	manager	mgr_salary
2	Bala	120	Asha	100
5	Esha	130	Bala	120

Key idea: Here an INNER JOIN is right: the top boss has no manager to compare with, so that row should not appear.

Solution 19 Recursive CTE

```
WITH d AS (
  SELECT CAST('2026-05-01' AS DATE) AS dt
  UNION ALL
  SELECT DATEADD(DAY, 1, dt)
  FROM d
  WHERE dt < '2026-05-05'
)
SELECT dt FROM d ORDER BY dt;
```

Expected output

dt
2026-05-01
2026-05-02
2026-05-03
2026-05-04
2026-05-05

Key idea: The condition tests the date going in, so the last date produced is 05-05 exactly. For more than 100 days add OPTION (MAXRECURSION 0).

Solution 20 Recursive CTE

```
WITH t AS (
  SELECT emp_id, name, 1 AS lvl,
         CAST(name AS VARCHAR(200)) AS path
  FROM OrgTree
  WHERE manager_id IS NULL
  UNION ALL
  SELECT o.emp_id, o.name, t.lvl + 1,
         CAST(t.path + ' > ' + o.name AS VARCHAR(200))
  FROM OrgTree o
  JOIN t ON o.manager_id = t.emp_id
)
SELECT emp_id, name, lvl, path
FROM t
ORDER BY lvl, emp_id;
```

Expected output

emp_id	name	lvl	path
1	Meera	1	Meera
2	Karan	2	Meera > Karan
3	Zoya	2	Meera > Zoya
4	Ishaan	3	Meera > Karan > Ishaan
5	Pooja	4	Meera > Karan > Ishaan > Pooja

Key idea: The anchor is the top boss. Each round adds the people who report to the previous round and extends level and path.